`findPartition`, returning its result (line #5).

This new implementation of `findPartition`, as we already mentioned, is not running in constant-time anymore: we will have as many recursive calls as the height of the partition's tree. Since we can't make any assumption on the trees so far, this means that we possibly have a number of calls proportional to the number of elements in the universe `U` - although this is the worst case and, on average, we can expect far better performance.

It might seem that, so far, we have only made our data structure's performance worse, apparently. We need to define our new implementation of the `merge` operation, shown in listing 5.7, to see the advantage of using trees.

By comparing the two implementations you can immediately see this is simpler, although only the last few lines changed. The good news is that we no longer need to iterate through a list of elements! To merge two partitions, we simply need to get to both trees' root, and then set one root as the parent of the other. We still need to find those tree roots, though.

The new line we added only require constant time, so the method runtime is dominated by the two calls to `findPartition`. As we have seen, they require time proportional to the height of the tree they are called on, and in the worst case this can still be linear. However, in the average case, and especially in the early stages after initialization, we know the height of the trees will be much smaller.

Listing 5.7 Tree-based solution, merge